

Normalization bounds in rudimentary linear lambda calculus [★]

Maria Emilia Maietti¹, Valeria de Paiva², and Eike Ritter³

¹ Dipartimento di Matematica Pura ed Applicata, Università di Padova, Italy.

e-mail: maietti@math.unipd.it

² Xerox Palo Alto Research Center - USA e-mail: paiva@xerox.parc.com

³ School of Computer Science, University of Birmingham, UK

E.Ritter@cs.bham.ac.uk

Abstract. Surprisingly we show that in rudimentary intuitionistic linear lambda calculus there are no linear bounds for normalizing terms under substitutions when commuting conversions are considered. We show that such a bound exists if we only count β -reductions or consider evaluation to values of closed terms.

Key words: linear logic, lambda calculus, complexity, normalization.

1 Introduction

Intuitionistic Linear logic [GL87, Laf88] and its associated *linear* lambda-calculus have been used in many ways in the analysis of functional programming languages [Wad90, Laf88]. The literature shows different lambda calculi associated to Intuitionistic Linear Logic [BBdH93, Ben95, BP97] but they all coincide on Rudimentary Linear Logic, the fragment of multiplicative Intuitionistic Linear Logic without modalities.

In this paper we study linear complexity bounds for reduction in the type theory *RL* associated to Rudimentary Linear Logic. We consider only time complexity, not space complexity (like in [Hof99]). We use the number of reduction steps a term might undergo as a measure for the time it takes to compute its normal form. The number of reduction steps is a sensible time complexity measure because we can implement these reductions in constant time. This is clear for commuting reductions, as they are local reduction rules, *i.e.*, affecting only the top of the syntax tree. Although the β -rule is global, in the sense that the effects of substitution cannot be limited to a fixed number of levels in the syntax tree, via annotations in the syntax tree for the occurrence of the bound variable it is possible to implement substitution and hence β -reduction in constant time. This is impossible in the simply-typed λ -calculus, as we might have arbitrarily many instances of the bound variables.

[★] Research partially supported by EPSRC-grant GR/L28296 under the title “The eXplicit Substitution Linear Abstract Machine”.

As linear variables occur only once in a *RLL* term, it is easy to see that all β -reductions reduce the size of the term. A direct consequence of this fact is the well-known observation that the number of possible β -redexes is linear in the size of the term. As the standard evaluation strategies for functional programming use only β -reductions, the linear bound on possible β -reductions also exists for these strategies.

If we consider linear implication only, we can show that for any term the number of possible reductions (after substitution of free variables) is linear in the number of normalization steps of substituted terms, with coefficients corresponding to virtual redexes and bounded by the size of the types of the variables. This result relies on a strong Church-Rosser property, which states that if $t \xrightarrow{1} t_1$ and $t \xrightarrow{1} t_2$ in one step and $t_1 \neq t_2$, then there exists a term s such that $t_1 \xrightarrow{1} s$ and $t_2 \xrightarrow{1} s$ in *one step*. A similar result holds also when we consider evaluation strategies to values, instead of reduction to normal form, for open terms in normal form after multiple substitutions with values in normal form. In this case the linear bound exists even for a calculus with tensor products and unit. We replace the number of reduction steps by the number of evaluation steps of a closed term. For closed terms evaluation we use only β -reductions, and the analogous of the strong Church-Rosser property then holds trivially, as the evaluation strategy is deterministic.

The picture changes when we consider all reductions (not only the β ones) for the calculus *RLL* with tensor products and their unit. Then we must consider not only β -reductions, which correspond to simplification of introduction-elimination pairs in linear logic, but also *commuting reductions*, which only rearrange the term. Some of these commuting reductions give rise to new β -redexes, hence they need to be executed both to obtain normal forms and in order to decide equality by reduction.

We show in this paper that the commuting reductions cause the complexity results for tensor products and units to differ significantly from those for linear implication only. Indeed, there is no even linear dependency of the number of possible reductions on the size of the term. We give an example in this paper where this dependency is in fact quadratic. Moreover, the number of possible reductions in the terms after substitutions is not linear in the number of normalization steps of substituted terms both with and without commuting reductions.

In conclusion we get the following table if we consider linear bound of normalization steps under substitutions with respect to the size or to the number of normalization steps of substituted terms:

linear bound in w.r.t.	<i>RLL</i>	<i>RLL</i> $_{-c}$	<i>LIMP</i>
size	no	yes	yes
red.steps	no	no	yes

where *RLL* $_{-c}$ is the system obtained from *RLL* by not considering commuting reductions.

Note that these commuting conversions are relevant when one considers reductions not only of closed terms but also of open terms. Linearity ensures that

β -reduction can be executed even for open terms without duplicating redexes. In functional programming this process is called inlining and is a standard optimization. The above bounds for reduction imply that inlining is always an optimization when we consider only linear implication, but may increase the number of required steps for a particular evaluation strategy when we consider also tensor products and units. In this case only certain commuting reductions should be executed.

A very different way of implementing a term calculus for linear logic is given by proof nets (first introduced in [Gir87]). In particular, they do not have any commuting conversions for the tensor product. The reason is that the re-wiring of links during β -reduction is a more powerful operation than substitution in the λ -calculus. Hence it is possible to obtain linear complexity bounds for reduction of proof nets for linear logic with linear implication and tensor products. But then we need to find translations from the λ -terms into proofnets and vice versa which preserve normal forms and have linear complexity.

We first recall the rudimentary linear lambda calculus *RLL* to fix our notation and we describe its reduction version suitable for our purposes. Then we discuss all the counterexamples for the various analogous linear bounds in *RLL* with respect to the normalization steps and the size of substituted terms. Finally we come back to *RLL* and after recalling that the usual evaluation strategy is linear on the size, using the previous machinery we provide a linear bound of substitution with values for programs in normal form.

2 Rudimentary linear lambda calculus

We start by describing the type theory associated to the fragment of Intuitionistic Linear Logic without modalities (or exponentials, in Girard's terminology). This fragment of Intuitionistic Linear Logic is sometimes called Rudimentary Linear Logic and we call *RLL* the corresponding linear type theory.

The type theory *RLL* has judgements of the form $\Delta \vdash t : A$ where Δ consists of a multiset of linear variable declarations of the form $x : A$ and no variable is repeated. The set of types we work with is

$$A ::= A \multimap A \mid A \otimes A \mid I$$

Raw terms are defined inductively by:

$$\begin{aligned} m, n ::= & x \mid \lambda x^A. m \mid m \otimes n \mid \bullet \\ & \mid m(n) \mid \text{let } m \text{ be } x_1 \otimes x_2 \text{ in } n \mid \text{let } m \text{ be } \bullet \text{ in } n \end{aligned}$$

Variables, λ -abstraction, application and tensor terms are as usual. Because the two kinds of **let**-expressions behave so similarly, we sometimes write **let** M **be** p **in** N to cover both, where p is either $x_1 \otimes x_2$ or $\bullet$.

We have three kinds of equations, β equations, η -equations and commuting conversion equations. The last kind of equations, familiar in the setting of linear lambda-calculi with sums, arises due to the form of η -rules for the tensor product

$\frac{\Delta, z : A \vdash b : B}{\Delta \vdash \lambda z^A. b : A \multimap B}$		$\frac{a : A \vdash a : A \quad \Delta_1 \vdash f : A \multimap B \quad \Delta_2 \vdash a : A}{\Delta \vdash f(a) : B}$
$\frac{\Delta_1 \vdash a : A \quad \Delta_2 \vdash b : B}{\Delta \vdash a \otimes b : A \otimes B}$	$\frac{\Delta_1 \vdash d : A \otimes B \quad z_1 : A, z_2 : B, \Delta_2 \vdash c : C}{\Delta \vdash \text{let } d \text{ be } z_1 \otimes z_2 \text{ in } c : C}$	
$\frac{}{\emptyset \vdash \bullet : I}$	$\frac{\Delta_1 \vdash m : I \quad \Delta_2 \vdash c : C}{\Delta \vdash \text{let } m \text{ be } \bullet \text{ in } c : C}$	

Where applicable Δ_1, Δ_2 are disjoint and Δ is a permutation of Δ_1, Δ_2 .

Table 1. The typing rules of *RLL*

and its unit. For the presentation of these equations we use contexts-with-holes, written $C[_]$. Contexts-with-holes are given by the grammar

$$C[_] ::= _ \mid \lambda z^A. C[_] \mid C[_](m) \mid m(C[_]) \\ \mid C[_] \otimes m \mid m \otimes C[_] \mid \text{let } C[_] \text{ be } p \text{ in } n \mid \text{let } m \text{ be } p \text{ in } C[_]$$

Note that this definition implies that there is exactly one occurrence of the symbol $_$ in a context-with-hole $C[_]$. The term $C[m]$ denotes the replacement of $_$ in $C[_]$ by m with the possible capture of free variables. The possibility of variable capture is the difference between the replacement of $_$ and substitution for a free variable: if $C[_]$ is the context-with-hole $(\lambda z^A. _)$, then $C[z] = \lambda z^A. z$, whereas $(\lambda z^A. b)[z/b] = \lambda c^A. z$.

The equations for *RLL* are given by:

β -equations:

$$\begin{aligned} (\lambda z : A. m)(n) &= m[n/z] \\ \text{let } m \otimes n \text{ be } z_1 \otimes z_2 \text{ in } r &= r[m/z_1, n/z_2] \\ \text{let } \bullet \text{ be } \bullet \text{ in } m &= m \end{aligned}$$

η -equations:

$$\begin{aligned} \lambda a : A. m(a) &= m \\ \text{let } m \text{ be } x \otimes y \text{ in } x \otimes y &= m \\ \text{let } m \text{ be } \bullet \text{ in } \bullet &= m \end{aligned}$$

Commuting conversions:

$$\begin{aligned} \text{let } m \text{ be } \bullet \text{ in } C[n] &= C[\text{let } m \text{ be } \bullet \text{ in } n] \\ \text{let } m \text{ be } z_1 \otimes z_2 \text{ in } C[n] &= C[\text{let } m \text{ be } z_1 \otimes z_2 \text{ in } n] \end{aligned}$$

provided that in the last conversion C does not bind or contain z_1 and z_2 .

We recall that a sound and complete semantics for *RLL* is provided by *autonomous* (or monoidal closed) categories. Moreover *RLL* is also the internal language [MRA93,KO99,MMdPR01] for autonomous categories.

2.1 Reduction rules for *RLL*

To perform computations in *RLL* we need to agree on which reduction rules to consider, given that we want a confluent and strongly normalizing calculus. If we simply orient the equations into reduction rules we can easily get a non-normalizing system because of the circularity of commuting reductions like

$$\text{let } r \text{ be } z \otimes w \text{ in } (\text{let } m \text{ be } x \otimes y \text{ in } n) \xrightarrow{1} \text{let } m \text{ be } x \otimes y \text{ in } (\text{let } r \text{ be } z \otimes w \text{ in } n)$$

Instead we are interested in describing the minimal set of reduction rules such that the corresponding normal forms belong to the equivalence classes of the calculus with only β -reductions modulo commuting reductions. As traditional in functional programming we do not consider η -reductions. The reduction rules we consider are as follows:

Red

$$\begin{array}{ll} (\lambda z : A. m)(n) & \xrightarrow{1} m[n/z] \\ \text{let } m \otimes n \text{ be } z_1 \otimes z_2 \text{ in } r & \xrightarrow{1} r[m/z_1, n/z_2] \\ \text{let } \bullet \text{ be } \bullet \text{ in } m & \xrightarrow{1} m \\ (\text{let } w \text{ be } p \text{ in } f)(a) & \xrightarrow{1} \text{let } w \text{ be } p \text{ in } f(a) \\ \text{let } (\text{let } w \text{ be } p' \text{ in } c) \text{ be } p \text{ in } f & \xrightarrow{1} \text{let } w \text{ be } p' \text{ in } (\text{let } c \text{ be } p \text{ in } f) \end{array}$$

Using standard techniques [Gha95] we can prove:

Proposition 1. *The type theory *RLL* considered with the reduction rules **Red** is confluent and strongly normalizing.*

Definition 1. *Given an *RLL* term t we define $\nu(t)$ as the length of the longest reduction sequence of the strongly normalizing term t .*

Definition 2. *For every term t in *RLL* we call $\text{nf}(t)$ the normal form of t with respect to the reduction relation $\xrightarrow{1}$.*

Recall that a term is in *introductory normal form* if the term is in normal form and the last rule used in its derivation was an introduction rule. In *RLL* this means that it is a tensor term or a lambda term or $\bullet$. Thus we can list the *RLL* normal forms as follows:

Proposition 2. *If f is a term in *RLL* in normal form then f is in introductory normal form or it is equal to one of the following terms:*

$$f \equiv \begin{cases} rlx_1 & \\ x_i(b_1) \dots (b_n) & \text{with } b_i \text{ in normal form} \\ \text{let } x_i(b_1) \dots (b_m) \text{ be } \bullet \text{ in } g & \text{with } b_i \text{ and } g \text{ in normal form} \\ \text{let } x_i(b_1) \dots (b_m) \text{ be } z_1 \otimes z_2 \text{ in } g & \text{with } b_i \text{ and } g \text{ in normal form} \end{cases}$$

Note that it is specific to purely linear lambda calculi that the size of a term (measured simply by counting its number of symbols) does not increase during β or commuting reductions.

Proposition 3. *If $t \xrightarrow{1} t'$ in RLL , then $size(t) \geq size(t')$.*

For the precise formulation of the bounds later on we need a definition of type size. It is as follows;

Definition 3. *The size Sz of a type in RLL is defined as follows:*

$$Sz(I) = 2 \quad Sz(A \multimap B) \equiv Sz(A) + Sz(B) + 2 \quad Sz(A \otimes B) \equiv Sz(A) + Sz(B) + 2$$

3 No Linear Bounds in RLL

We show in this section that we cannot find linear bounds in RLL both with and without commuting reductions for the number of steps of normalizing terms after substitutions with respect to the normalization steps of substituted terms.

3.1 Counterexamples with no commuting reductions

Note that in the absence of commuting reductions we cannot bound the maximum number of reduction steps after substitutions with respect to the normalization steps of substituted terms. Indeed we show here a counterexample in the case of absence of commuting reductions for the unit. We recall that with $RLL \multimap_c$ we mean RLL without commuting reductions.

Proposition 4. *The following term in $RLL \multimap_c$*

$$x : I \multimap I \vdash x^{I \multimap I}(\bullet) : I$$

has no reduction bound with respect to the maximum number of normalization steps of all the possible substituted terms.

Proof. We define the following family of terms in normal form by induction

$$\begin{aligned} t_0 &\equiv \text{let } w_0 \text{ be } \bullet \text{ in } \bullet : I \\ t_{n+1} &\equiv (\text{let } w_{n+1} \text{ be } \bullet \text{ in } \lambda w_n. t_n)(\bullet) : I \end{aligned}$$

Hence if we consider $x(\bullet)[\lambda w_n. t_n/x]$ we can now prove that $\nu((\lambda w_n. t_n)(\bullet)) \geq 2 + \nu(\lambda w_{n-1}. t_{n-1})$ by induction and then $\nu(x(\bullet)[\lambda w_n. t_n/x]) = \nu((\lambda w_n. t_n)(\bullet)) \geq 2(n+1)$.

Moreover we show also a counterexample in the absence of commuting reductions for tensor:

Proposition 5. *The following term in $RLL \multimap_c$*

$$z : (A \otimes B) \multimap (A \otimes B) \otimes (B \otimes A), \quad x : A, \quad y : B \vdash z(x \otimes y) : (A \otimes B) \otimes (B \otimes A)$$

has no reduction bound with respect to the maximum number of normalization steps of all the possible substituted terms.

Proof. Define inductively the following terms: given $u_n : A \otimes B$, $x_n : A$ and $y_n : B$

$$\begin{aligned} s_0 &\equiv u_0 \otimes y_0 \otimes x_0 \\ s_n &\equiv (\text{let } u_n \text{ be } x_{n-1} \otimes y_{n-1} \text{ in } \lambda u_{n-1}. s_{n-1})(x_n \otimes y_n) \end{aligned}$$

All s_i 's are normal forms provided that we do not have any commuting reduction. Note that $s_n[x \otimes y / u_n]$ reduces to $x_n \otimes y_n \otimes y \otimes x$ if n is odd and to $x \otimes y \otimes y_n \otimes x_n$ if n is even.

Now, if we consider $t_n \equiv \lambda u_n. s_n$, then the normal form of $z(x \otimes y)[t_n / z]$ is either $x_n \otimes y_n \otimes y \otimes x$ or $x \otimes y \otimes y_n \otimes x_n$. By induction we can prove that $\nu(z(x \otimes y)[t_n / z]) \geq 2n + 1$

3.2 Counterexamples of linear bounds with commuting reductions

For *RLL* with β and commuting reductions there are no linear bounds with respect to the the number of reductions in either the length of the longest reduction sequence nor the size of the term to be substituted.

Proposition 6. *The following RLL term*

$$z : (A \otimes B), u : (A \otimes B) \multimap (A \otimes B) \vdash \text{let } z \text{ be } x_0 \otimes y_0 \text{ in } (u(x_0 \otimes y_0)) : A \otimes B$$

does not have any linear bound with respect to maximum number of normalization steps of all the possible substituted terms.

Proof. Consider the following *RLL* terms in normal form of type $A \otimes B$: given $z_0 : A \otimes B$ and for $i \geq 1$ $z_i : A \otimes B \multimap A \otimes B$

$$\begin{aligned} u_1 &\equiv \text{let } z_0 \text{ be } x_1 \otimes y_1 \text{ in } (z_1(x_1 \otimes y_1)) \\ u_2 &\equiv \text{let } z_0 \text{ be } x_1 \otimes y_1 \text{ in let } (z_2(x_2 \otimes y_2)) \text{ be } x_3 \otimes y_3 \text{ in } (z_3(x_3 \otimes y_3)) \\ u_{n+1} &\equiv u_n \{ \text{let } (z_n(x_n \otimes y_n)) \text{ be } x_{n+1} \otimes y_{n+1} \text{ in } (z_{n+1}(x_{n+1} \otimes y_{n+1})) / z_n(x_n \otimes y_n) \} \end{aligned}$$

where $u_n \{ \text{let } (z_n(x_n \otimes y_n)) \text{ be } x_{n+1} \otimes y_{n+1} \text{ in } (z_{n+1}(x_{n+1} \otimes y_{n+1})) / z_n(x_n \otimes y_n) \}$ means that we substitute in u_n the expression $z_n(x_n \otimes y_n)$ with

$$\text{let } (z_n(x_n \otimes y_n)) \text{ be } x_{n+1} \otimes y_{n+1} \text{ in } (z_{n+1}(x_{n+1} \otimes y_{n+1}))$$

Then we can prove that for arbitrary n

$$\nu(\text{let } z \text{ be } x_0 \otimes y_0 \text{ in } (u(x_0 \otimes y_0))[u_n / z]) \geq n$$

Note that the strategy to reduce $\text{let } z \text{ be } x_0 \otimes y_0 \text{ in } (u(x_0 \otimes y_0))[u_n / z]$ is unique, namely at every step only the outermost *let*-reduction can be performed for a number of times equal to the numbers of *let* occurrences.

Having found a term for which the evaluation strategy after some substitutions is unique and with no linear bound with respect to the normalization steps of substituted terms means that there is no general best strategy to compute lambda terms after substitutions in linear time with respect to the normalization steps of substituted terms.

Moreover we can find a term which does not have a linear bound even with respect to the size:

Proposition 7. *The following RLL term*

$$z_0 : A \otimes B, z_1 : (A \otimes B) \multimap (A \otimes B), \dots, z_n : (A \otimes B) \multimap (A \otimes B) \vdash u_n : A \otimes B$$

defined as

$$\begin{aligned} u_0 &\equiv z_0 \\ u_n &\equiv \text{let } u_{n-1} \text{ be } x_n \otimes y_n \text{ in } (z_n(x_n \otimes y_n)) \end{aligned}$$

does have at least a quadratic complexity of reductions with respect to its size.

Proof. We can see that u_n reduces in n steps to

$$\text{let } z_0 \text{ be } x_1 \otimes y_1 \text{ in } u'_{n-1}[z_1(x_1 \otimes y_1)/z_0]$$

where u'_n is u_n with all bound variables x_i and y_i and z_i replaced by x_{i+1} and y_{i+1} and z_{i+1} respectively. If we apply this reduction recursively to $u'_{n-1}[z_1(x_1 \otimes y_1)/z_0]$, we obtain the longest possible reduction sequence. This implies that $\nu(u_n) \geq \frac{n(n+1)}{2}$. Since the size of u_n grows linearly, then there is in general no linear bound of $\nu(x[u_n/x])$ with respect to $\text{size}(u_n)$. However the shortest reduction sequence has length n and is obtained by reducing the outermost *let* commuting reduction first.

Since in the above counterexample the absence of linear bound with respect to the size exists if we consider the worst strategy, then there might be a room to optimize the strategy of evaluation, contrary to what we showed in prop. 6 for the complexity with respect to the number of normalization steps.

4 Linear bound in RLL modulo commuting reductions

In this section we establish a bound by counting only β -reduction in reducing any RLL-term to a normal form.

Definition 4. *In RLL we define a new reduction relation $\rightsquigarrow$ by $t \rightsquigarrow s$ by iff there exist terms $t_1, \dots, t_n$ and $s_1, \dots, s_m$ such that all t_j reduce to t_{j+1} and s_i to s_{i+1} via single-step commuting conversion reductions and $t_n = t$ and $s_1 = s$, and t reduces to s via exactly one β -reduction.*

Definition 5. *We call $\nu_{/cr}(t)$ the biggest number of β -reductions needed to reduce a RLL-term t to a RLL-normal form.*

Note that counting $\nu_{/cr}(t)$ is like counting the number of β -reductions of the equivalence class of terms in RLL modulo commuting reductions, where the reduction of an equivalence class is defined by reducing any representative term with exactly one β -reduction plus commuting reductions. This follows from the fact that in any equivalence class of terms non further reducible via β -reductions we have only one term in normal form with respect to RLL-reductions.

The key point is now that the strong Church-Rosser property holds for this notion of reduction:

Proposition 8. Assume t_1 and t_2 are RLL terms. Given $t \rightsquigarrow t_1$, $t \rightsquigarrow t_2$ and $t_1 \neq t_2$ then there exists a term t_3 such that $t_1 \rightsquigarrow t_3$ and $t_2 \rightsquigarrow t_3$.

Proof. We proceed by induction on the size of the term.

Hence again from this it easily follows that

Corollary 1. For every t in RLL, if $t \rightsquigarrow t'$ then

$$\nu_{/cr}(t) = 1 + \nu_{/cr}(t')$$

4.1 Virtual redexes

To establish bounds for substituting terms we need to keep track which redexes might be created during substitutions. We call those redexes *virtual redexes*. Their number depends on the type of the substitution variable.

Definition 6. For every RLL term f with the free variable x we define $V_x(f)$, the number of virtual redexes of f with respect to f , by induction over the structure of f by

$$\begin{aligned} V_x(x) &\equiv 0 \\ V_x(\lambda y.b) &\equiv V_x(b) \\ V_x(f(b)) &\equiv \begin{cases} V_x(f) & \text{if } x \text{ appears in } f \text{ but not in head position} \\ V_x(b) & \text{if } x \text{ appears in } b \\ \Sigma_{i=1}^{n+1}(sz(B_i) + 1) & \text{if } f \equiv x(b_1)(b_2)\dots(b_n), b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1, \dots, n+1. \end{cases} \\ V_x(\text{let } x \text{ be } z_1 \otimes z_2 \text{ in } h) &\equiv V_{z_1}(h) + V_{z_2}(h) + 1 \\ V_x(\text{let } f(b) \text{ be } z_1 \otimes z_2 \text{ in } h) &\equiv \begin{cases} V_x(h) & \text{if } x \text{ appears in } h \\ V_x(f(b)) & \text{if } x \text{ appears in } f(b) \text{ but not in head position} \\ \Sigma_{i=1}^{n+1}(sz(B_i) + 1) + (V_{z_1}(h) + V_{z_2}(h) + 1) & \text{if } f \equiv x(b_1)(b_2)\dots(b_n), b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1, \dots, n+1. \end{cases} \\ V_x(\text{let } x \text{ be } \bullet \text{ in } h) &\equiv 1 \\ V_x(\text{let } f(b) \text{ be } \bullet \text{ in } h) &\equiv \begin{cases} V_x(h) & \text{if } x \text{ appears in } h \\ V_x(f(b)) & \text{if } x \text{ appears in } f(b) \\ & \text{but not in head position} \\ \Sigma_{i=1}^{n+1}(sz(B_i) + 1) + 1 & \text{if } f \equiv x(b_1)(b_2)\dots(b_n), b \equiv b_{n+1} \\ & \text{where } b_i : B_i \text{ for } i = 1, \dots, n+1. \end{cases} \end{aligned}$$

Thanks to prop 2 we can establish links between the virtual redex of variables appearing in non-introductory normal forms and their subterms:

Proposition 9. For every $t \equiv f(a)$ or $t \equiv \text{let } f \text{ be } p \text{ in } d : D$ in normal form one of the following cases happens:

$$\begin{cases} V_x(f(a)) \geq V_x(f) + 1 + \text{sz}(A) & \text{if } t \equiv f(a) \text{ and } A \text{ is the type of } a \\ V_x(\text{let } f \text{ be } \bullet \text{ in } d) \geq V_x(f) + 1 & \text{if } t \equiv \text{let } f \text{ be } \bullet \text{ in } d \\ V_x(\text{let } f \text{ be } z_1 \otimes z_2 \text{ in } d) \geq & \text{if } t \equiv \text{let } f \text{ be } z_1 \otimes z_2 \text{ in } d \\ V_x(f) + V_{z_1}(d) + V_{z_2}(d) + 1 & \end{cases}$$

where x is the head variable of the normal form of f and A is the type of the variable x .

Proposition 10. For every variable $x : B$ appearing in a term f in normal form we have

$$V_x(f) \leq \text{sz}(B)$$

Proof. The proof goes by induction on the size of t .

Lemma 1. If f is a subterm of t , both in normal form, then

$$V_x(f) \leq V_x(t)$$

for all x appearing at least in f .

Proof. The proof goes by induction on the size of t .

Proposition 11. If f is a term in RLL in normal form then f is in introductory normal form or it is equal to one of the following terms:

$$f \equiv \begin{cases} x_1 & \\ x_i(b_1) \dots (b_n) & \text{with } b_i \text{ in normal form} \\ \text{let } x_i(b_1) \dots (b_m) \text{ be } \bullet \text{ in } g & \text{with } b_i \text{ and } g \text{ in normal form} \\ \text{let } x_i(b_1) \dots (b_m) \text{ be } z_1 \oplus z_2 \text{ in } g & \text{with } b_i \text{ and } g \text{ in normal form} \end{cases}$$

Proposition 12. For every $t \equiv f(a)$ or $t \equiv \text{let } f \text{ be } p \text{ in } d : D$ in normal form then one of the following case happens: for some i

$$\begin{cases} V_{x_i}(f(a)) \geq V_{x_i}(f) + 1 + \text{sz}(A) & \text{if } t \equiv f(a) \text{ and } A \text{ the type of } a \\ V_{x_i}(\text{let } f \text{ be } \bullet \text{ in } d) \geq V_{x_i}(f) + 2 & \text{if } t \equiv \text{let } f \text{ be } \bullet \text{ in } d \\ V_{x_i}(\text{let } f \text{ be } z_1 \otimes z_2 \text{ in } d) \geq & \text{if } t \equiv \text{let } f \text{ be } z_1 \otimes z_2 \text{ in } d \\ V_{x_i}(f) + V_{z_1}(d) + V_{z_2}(d) + 1 & \end{cases}$$

Note that the size decreases during reductions:

Lemma 2. If $t \rightsquigarrow t'$ then

$$\text{size}(t) \geq \text{size}(t')$$

On this we base part of the induction to prove the next theorems. The main theorem we want to prove provides a bound to normalize terms under substitutions. We will provide the proof of this theorem in the appendix.

Theorem 1. For every term t in RLL where $\bar{x} \equiv x_1, \dots, x_n$ include the free variables appearing in t then

$$\nu_{/cr}(t[\bar{s}/\bar{x}]) \leq \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\text{nf}(t)) + \nu_{/cr}(t)$$

5 Linear bounds for evaluation of closed terms

If we consider the standard evaluation of closed terms to suitable values instead of the reduction to a normal form, then its complexity with respect to the size of the evaluated term is linear, as we will show in this section. More precisely, we define values by

$$v ::= \bullet \mid \lambda x : A. f \mid v \otimes v$$

where f is a term with the only free variable $x : A$.

For closed terms t we then define a relation $t \Downarrow v$ inductively by

$$\begin{array}{c} \frac{}{v \Downarrow v} \qquad \frac{t \Downarrow \lambda x. t' \quad s \Downarrow v' \quad t'[v'/x] \Downarrow v}{ts \Downarrow v} \\[10pt] \frac{t_1 \Downarrow v_1 \quad t_2 \Downarrow v_2}{t_1 \otimes t_2 \Downarrow v_1 \otimes v_2} \quad \frac{t \Downarrow v_1 \otimes v_2 \quad s[v_1/x, v_2/y] \Downarrow v}{\text{let } t \text{ be } x \otimes y \text{ in } s \Downarrow v} \\[10pt] \frac{t \Downarrow \bullet \quad s \Downarrow v}{\text{let } t \text{ be } \bullet \text{ in } s \Downarrow v} \end{array}$$

The relation $\Downarrow$ is the usual evaluation relation familiar from functional programming. Because there is no weakening, the call-by-value and call-by-name variants are equivalent.

Note that the definition of the evaluation relation provides a reduction strategy to a value only through β -reductions and without commuting reductions.

As a consequence the complexity of the evaluation is linear with respect to the size of the term under evaluation.

Proposition 13. *Let t be any closed term. Then the number of steps to evaluate t to v according to the above evaluation relation is at most $\text{size}(t)$ steps.*

Proof. First of all we can easily prove by induction on the number of steps to derive $t \Downarrow v$ that the evaluation of a term through its value is done through only β -reduction steps. Then, if we define $\nu_{\text{val}}(t)$ to be the number of steps to reduce t to a value according to the above evaluation, by induction on the size of t we can derive that $\nu_{\text{val}}(t) \leq \text{size}(t)$.

We obtain stronger bounds for evaluation by relating them to the bounds obtained for reductions modulo commuting reductions.

Lemma 3. *Assume $t \Downarrow v$ via n applications of inference rules not counting the rule $v \Downarrow v$. Then t reduces to v in n steps.*

Hence we obtain as a direct corollary of theorem 1:

Theorem 2. *For every term t of RLL in normal form with free variables among $\bar{x} \equiv x_1, \dots, x_n$ substituted with terms $\bar{s} \equiv s_1, \dots, s_n$ in normal form we have that*

$$\nu_{\text{val}}(t[\bar{s}/\bar{x}]) \leq \sum_{i=0}^n V_{x_i}(t)$$

Proof. The theorem follows from theorem 1 since $\nu_{val}(t[\overline{s}/\overline{x}])$ is bounded by the number of step to reduce the term modulo commuting reductions since no commuting reductions are needed to evaluate the term.

Note that this bound for evaluation of f in normal form after substitutions is tighter than the corresponding one with respect to the size.

6 Conclusion

The non-existence of linear bounds in rudimentary linear lambda calculus for normalizing terms under substitution shows that inlining has no linear complexity in *RLL* unless we restrict to its linear implicational fragment and for it we give a linear bound tighter than the size. In future work we want to explore the complexity of some variants of RLL with linear recursion along the lines of the work in [Hof99,AS00]. We also would like to investigate the translations between proof nets and linear λ -terms with respect of their complexity.

Acknowledgments

We would like to thank Silvio Valentini and Arnaud Fleury for discussions on the subject of this paper.

References

- [AS00] K. Aehlig and H. Schwichtenberg. A syntactical analysis of non-size-increasing polynomial time computation. In *Lics 2000*, IEEE, Computer Society Press, 2000.
- [BBdH93] N. Benton, G. Bierman, V. de Paiva, and M. Hyland. A term calculus for intuitionistic linear logic. In *Proc. of Typed Lambda Calculus and Applications.*, volume 664 of *Lecture Notes in Computer Science*, pages 75–90. Springer Verlag, 1993.
- [Ben95] Nick Benton. A mixed linear and non-linear logic: Proofs, terms and models. In *Proceedings of Computer Science Logic '94, Kazimierz, Poland*. Lecture Notes in Computer Science No. 933, Berlin, Heidelberg, New York, 1995.
- [BP97] A. Barber and G. Plotkin. Dual intuitionistic linear logic. Technical report, LFCS, University of Edinburgh, 1997.
- [Gha95] N. Ghani. *Adjoint Rewriting*. PhD thesis, University of Edinburgh, 1995. Published as CST-122-95 and ECS-LFCS-95-339.
- [Gir87] J.Y. Girard. Linear logic. *Theoretical Computer Science*, 50:1–101, 1987.
- [GL87] J.Y. Girard and Y. Lafont. Linear logic and lazy computation. In *TAP-SOFT '87*, volume 250 of *Lecture Notes in Computer Science*, pages 52–66, 1987.
- [Hof99] M. Hofmann. Linear types and non-size-increasing polynomial time computation. In *Proc. of LICS '99*, IEEE, Computer Society Press, 1999.

- [KO99] T. W. Koh and C.-H. L. Ong. Explicit substitution internal languages for autonomous and *-autonomous categories (preliminary version). In *CTCS '99*, volume 29 of *Electronic Notes in Theoretical Computer Science*, 1999.
- [Laf88] Y. Lafont. The linear abstract machine. In *International Joint Conference on Theory and Practice of Software Development. (Pisa, 1987)*, volume 59 of *Theoretical Computer Science*, pages 157–180, 1988.
- [MMdPR01] M. Maietti, P. Maneggia, V. de Paiva, and E. Ritter. Linear type theories and their models. *Preprint-University of Padova*, 1:10 January, 2001.
- [MRA93] I. Mackie, L. Roman, and S. Abramsky. An internal language for autonomous categories. *Applied Categorical Structures*, 1:311–343, 1993.
- [Wad90] P. Wadler. Linear types can change the world! In M. Broy and C. Jones, editors, *Programming Concepts and Methods*, 1990.

A Proof of the main theorem

This appendix contains the proof of the main theorem.

Theorem 3. *For every term t in RLL where $\bar{x} \equiv x_1, \dots, x_n$ include the free variables appearing in t then*

$$\nu_{/cr}(t[\bar{s}/\bar{x}]) \leq \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\mathbf{nf}(t)) + \nu_{/cr}(t)$$

Proof. The proof goes by induction on $sz(t[\bar{s}/\bar{x}])\omega + \nu_{/cr}(t)\omega^2$. Note that we abbreviate “inductive hypothesis” with ind. hyp. We can distinguish the following cases:

1. t is not in normal form, then if $t \rightsquigarrow t''$

$$\begin{aligned} \nu_{/cr}(t[\bar{s}/\bar{x}]) &= 1 + \nu_{/cr}(t'[\bar{s}/\bar{x}]) \\ &\quad \text{by cor.1} \\ &\leq 1 + \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\mathbf{nf}(t')) + \nu_{/cr}(t') \\ &\leq \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\mathbf{nf}(t)) + \nu_{/cr}(t) \\ &\quad \text{since } \nu_{/cr}(t) = \nu_{/cr}(t') + 1 \text{ by cor.1 and } \mathbf{nf}(t) = \mathbf{nf}(t') \end{aligned}$$

2. t is in normal form but some s_i is not in normal form, then if $s_j \rightsquigarrow s'_j$

$$\begin{aligned} \nu_{/cr}(t[\bar{s}/\bar{x}]) &= 1 + \nu_{/cr}(t[s_i/x_i]_{i \neq j}[s'_j/x_j]) \\ &\quad \text{by cor.1} \\ &\leq \sum_{i=0, i \neq j}^n \nu_{/cr}(s_i) + \nu_{/cr}(s'_j) \sum_{i=0, i \neq j}^n V_{x_i}(\mathbf{nf}(t)) + \nu_{/cr}(t) \\ &\quad \text{by ind. hyp. on } size(t[s_i/x_i]_{i \neq j}[s'_j/x_j]) \\ &= \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\mathbf{nf}(t)) + \nu_{/cr}(t) \\ &\quad \text{since } \nu_{/cr}(s_j) = 1 + \nu_{/cr}(s'_j) \text{ by cor.1} \end{aligned}$$

3. t and every s_i are in normal form: in this case in turn we have a list of subcases
 - (a) If $t \equiv x_1$ the thesis is obviously valid.

(b) If $t \equiv \lambda w.g$ then by definition of substitution and normal form

$$\nu_{/cr}((\lambda w.g)[\bar{s}/\bar{x}]) = \nu_{/cr}(g([\bar{s}/\bar{x}]))$$

and by ind. hyp. on $size(g[\bar{s}/\bar{x}])$

$$\nu_{/cr}(g([\bar{s}/\bar{x}])) \leq \sum_{i=0}^n \nu_{/cr}(s_i) + \sum_{i=0}^n V_{x_i}(\text{nf}(g)) + \nu_{/cr}(g)$$

and since $V_{x_i}(\text{nf}(g)) = V_{x_i}(\text{nf}(\lambda w.g))$ and $\nu_{/cr}(g) = \nu_{/cr}(\lambda w.g)$ we conclude.

(c) If $t \equiv a \otimes b$ then

$$\nu_{/cr}(a \otimes b([\bar{s}/\bar{x}])) = \nu_{/cr}(a[\bar{s}'/\bar{x}']) + \nu_{/cr}(b[\bar{s}''/\bar{x}''])$$

Then by ind. hyp. on $size(a[\bar{s}'/\bar{x}'])$

$$\nu_{/cr}(a[\bar{s}'/\bar{x}']) \leq \sum_{i'=0}^{n_1} \nu_{/cr}(s'_{i'}) + \sum_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(a)) + \nu_{/cr}(a)$$

and the same holds for b . Hence

$$\begin{aligned} \nu_{/cr}(a \otimes b[\bar{s}/\bar{x}])) &\leq \sum_{i'=0}^{n_1} \nu_{/cr}(s'_{i'}) + \sum_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(a)) + \nu_{/cr}(a) + \\ &\quad \sum_{i''=0}^{n_2} \nu_{/cr}(s''_{i''}) + \sum_{i''=0}^{n_2} V_{x_{i''}}(\text{nf}(b)) + \nu_{/cr}(b) \end{aligned}$$

We then conclude since $\nu_{/cr}(a \otimes b) = \nu_{/cr}(a) + \nu_{/cr}(b)$, $V_{x'_{i'}}(\text{nf}(a)) = V_{x'_{i'}}(\text{nf}(a) \otimes \text{nf}(b))$, $V_{x''_{i''}}(\text{nf}(b)) = V_{x''_{i''}}(\text{nf}(a) \otimes \text{nf}(b))$ and $\text{nf}(a) \otimes \text{nf}(b) = \text{nf}(a \otimes b)$.

(d) If $t \equiv f(a)$ and $\text{nf}(f[\bar{s}'/\bar{x}']) = \text{let } c_i \text{ be } p_i \text{ in } d$ then

$$\begin{aligned} \nu_{/cr}(f(a)[\bar{s}/\bar{x}])) &= \nu_{/cr}((f[\bar{s}'/\bar{x}'])(a[\bar{s}''/\bar{x}''])) \\ &= \nu_{/cr}(f[\bar{s}'/\bar{x}'] + \nu_{/cr}((\text{let } c_i \text{ be } p_i \text{ in } d)(a[\bar{s}''/\bar{x}'']))) \\ &\quad \text{by cor.1} \\ &\leq \sum_{i'=0}^{n_1} V_{x'_{i'}}(\text{nf}(f)) + \nu_{/cr}(\text{let } c_i \text{ be } p_i \text{ in } d(a[\bar{s}''/\bar{x}'']))) \\ &\quad \text{by ind. hyp. on } size(f[\bar{s}'/\bar{x}']) \text{ since } \nu_{/cr}(f) = \nu_{/cr}(s_i) = 0 \\ &= \sum_{i'=0}^{n_1} V_{x'_{i'}}(\text{nf}(f)) + \nu_{/cr}(d(a[\bar{s}''/\bar{x}'']))) \\ &\quad \text{since let } c_i \text{ be } p_i \text{ in } d \text{ is in normal form} \end{aligned}$$

Now there are two subcases provided that d is in introductory normal form or not.

If d is not introductory then (recalling that $\text{nf}(f(a)) = f(a)$)

$$\begin{aligned} \nu_{/cr}(f(a)[\bar{s}/\bar{x}])) &\leq \sum_{i'=0}^{n_1} V_{x'_{i'}}(\text{nf}(f)) + \nu_{/cr}(d(a[\bar{s}''/\bar{x}'']))) \\ &= \sum_{i'=0}^{n_1} V_{x'_{i'}}(\text{nf}(f)) + \nu_{/cr}(a[\bar{s}''/\bar{x}'']))) \\ &\quad \text{since } \nu_{/cr}(d(a[\bar{s}''/\bar{x}'']))) = \nu_{/cr}(d) + \nu_{/cr}(a[\bar{s}''/\bar{x}'']))) \\ &\quad \text{and } \nu_{/cr}(d) = 0 \\ &\leq \sum_{i'=0}^{n_1} V_{x'_{i'}}(\text{nf}(f)) + \sum_{i''=0}^{n_2} V_{x''_{i''}}(\text{nf}(a)) \\ &\quad \text{by ind.hyp on } size(a[\bar{s}''/\bar{x}'']) \text{ and } a \text{ in normal form} \\ &\leq \sum_{i=0}^n V_{x_i}(\text{nf}(f(a))) \\ &\quad \text{by lemma 1 and by prop.12} \\ &\quad \text{stating that for some } k \ V_{x_k}(f(a)) \geq 2 + V_{x_k}(f) \end{aligned}$$

If $d \equiv \lambda w.g$ then we proceed as follows:

$$\begin{aligned}
\nu_{/cr}(f(a)[\bar{s}/\bar{x}]) &\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(f)) + \nu_{/cr}((\lambda w.g)(a)[\bar{s}''/\bar{x}'']) \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(f)) + 1 + \nu_{/cr}(g[a[\bar{s}''/\bar{x}'']/w]) \\
&\quad \text{by cor.1} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(f)) + 1 + V_w(g) + \nu_{/cr}(a[\bar{s}''/\bar{x}'']) \\
&\quad \text{by ind. hyp. on } \text{size}(g[a[\bar{s}''/\bar{x}'']/w]) \text{ since } \nu_{/cr}(g) = \nu_{/cr}(s_i) = 0 \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(f)) + 1 + sz(A) + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\text{nf}(a)) \\
&\quad \text{supposed } A \text{ the type of } a \text{ by prop.10 and by ind. hyp. on } \text{size}(a[\bar{s}''/\bar{x}'']) \\
&\quad \text{since } \nu_{/cr}(a) = 0 \\
&\leq \Sigma_{i=0}^n V_{x_i}(\text{nf}(f(a))) \\
&\quad \text{since by prop.12 for some } j \ V_{x_j}(f(a)) \geq V_{x_j}(f) + 1 + sz(A)
\end{aligned}$$

(e) If $t \equiv \text{let } w \text{ be } p \text{ in } h$ and $\text{nf}(w[\bar{s}'/\bar{x}']) = \text{let } c_i \text{ be } q_i \text{ in } d$ with d not a let-expression then

$$\begin{aligned}
\nu_{/cr}(\text{let } w \text{ be } p \text{ in } h[\bar{s}/\bar{x}]) &= \nu_{/cr}(\text{let } w[\bar{s}'/\bar{x}'] \text{ be } p \text{ in } h[\bar{s}''/\bar{x}'']) \\
&= \nu_{/cr}(w[\bar{s}'/\bar{x}']) + \nu_{/cr}(\text{let } (\text{let } c_i \text{ be } q_i \text{ in } d) \text{ be } p \text{ in } h[\bar{s}''/\bar{x}'']) \\
&\quad \text{by cor.1} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + \nu_{/cr}(\text{let } c_i \text{ be } q_i \text{ in } (\text{let } d \text{ be } p \text{ in } h[\bar{s}''/\bar{x}''])) \\
&\quad \text{by ind. hyp. on } \text{size}(w[\bar{s}'/\bar{x}']) \text{ and cor.1} \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + \nu_{/cr}(\text{let } d \text{ be } p \text{ in } h[\bar{s}''/\bar{x}''])
\end{aligned}$$

Now there are several subcases depending on d . Firstly, assume that d is not in introductory normal form. Then we have

$$\begin{aligned}
\nu_{/cr}(\text{let } w \text{ be } p \text{ in } h[\bar{s}/\bar{x}]) &= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + \nu_{/cr}(h[\bar{s}''/\bar{x}'']) \\
&\quad \text{by assumption the let-expressions are in normal form} \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\text{nf}(h)) \\
&\quad \text{by ind. hyp. on } \text{size}(h[\bar{s}''/\bar{x}'']) \text{ being } w, h, s_i \text{ in normal form} \\
&\leq \Sigma_{i=0}^n V_{x_i}(\text{nf}(\text{let } w \text{ be } p \text{ in } h)) \\
&\quad \text{by lemma 1 and by prop.12} \\
&\quad \text{stating that for some } j \ V_{x_j}(\text{let } w \text{ be } p \text{ in } h) \geq 2 + V_{x_j}(w)
\end{aligned}$$

For the second subcase, assume that $p = \bullet$ and $d = \bullet$. In this case we have

$$\begin{aligned}
\nu_{/cr}(\text{let } w \text{ be } p \text{ in } h[\bar{s}/\bar{x}]) &= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + 1 + \nu_{/cr}(h[\bar{s}''/\bar{x}'']) \\
&\leq \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + 1 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\text{nf}(h)) \\
&\leq \Sigma_{i=0}^n V_{x_i}(\text{nf}(\text{let } w \text{ be } p \text{ in } h)) \\
&\quad \text{by lemma 1 and proposition 12}
\end{aligned}$$

For the final subcase, assume that $p = a \otimes b$ and $d = t \otimes s$. Then we have

$$\begin{aligned}
\nu_{/cr}(\text{let } w \text{ be } p \text{ in } h[\bar{s}/\bar{x}]) &= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + 1 + \nu_{/cr}(h[t/a][s/b][\bar{s}''/\bar{x}'']) \\
&= \Sigma_{i'=0}^{n_1} V_{x_{i'}}(\text{nf}(w)) + 1 + \Sigma_{i''=0}^{n_2} V_{x_{i''}}(\text{nf}(h)) + V_a h + V_b(h) \\
&\leq \Sigma_{i=0}^n V_{x_i}(\text{nf}(\text{let } w \text{ be } p \text{ in } h)) \\
&\quad \text{by lemma 1 and proposition 12}
\end{aligned}$$